Cahier des charges : Gestion d'un système de livraison

1. Contexte et objectifs

Dans le cadre du module Programmation Orientée Objet en Java, vous devez développer en binôme une application de gestion d'un système de livraison. Ce projet a pour objectif de mettre en pratique les notions vues en cours, notamment :

- Les classes et les objets
- Les attributs et les méthodes
- Les constructeurs
- L'encapsulation
- Les relations entre classes
- L'utilisation des collections
- L'organisation d'un programme Java en plusieurs classes

L'application devra permettre de gérer les clients, les commandes, les livreurs ainsi que les livraisons.

Objectifs pédagogiques

À travers ce projet, vous devrez être capables de

- Concevoir une application orientée objet en Java
- Modéliser un problème réel sous forme de classes
- Manipuler des objets liés entre eux
- Développer une application console structurée
- Travailler en binôme sur un projet court
- Documenter et présenter votre travail.

2. Organisation du projet

- Durée **:** 2 à 3 semaines.
- Travail : en binôme
- Langage : Java
- Type d'application : application console avec menu interactif
- Encadrement : points réguliers en séance avec l'enseignant

3. Fonctionnalités obligatoires

Chaque binôme doit implémenter au minimum les fonctionnalités suivantes :

3.1. Gestion des clients

L'application doit permettre de :

- ajouter un client
- supprimer un client
- modifier les informations d'un client
- afficher la liste des clients

- rechercher un client par identifiant ou nom

Chaque client peut contenir les informations suivantes :

- identifiant
- nom
- prénom
- téléphone
- adresse
- email

3.2. Gestion des livreurs

L'application doit permettre de :

- ajouter un livreur
- supprimer un livreur
- modifier les informations d'un livreur
- afficher la liste des livreurs
- rechercher un livreur par identifiant ou nom

Chaque livreur peut contenir les informations suivantes :

- identifiant
- nom
- prénom
- téléphone
- véhicule utilisé

3.3. Gestion des commandes

L'application doit permettre de :

- créer une commande pour un client
- supprimer une commande
- afficher la liste des commandes
- rechercher une commande par identifiant

Chaque commande peut contenir les informations suivantes :

- identifiant
- client concerné
- description de la commande
- date de commande
- statut de la commande
  Exemples de statuts :
- en attente
- en préparation
- en livraison
- livrée

3.4. Gestion des livraisons

L'application doit permettre de :

- affecter une commande à un livreur
- enregistrer la livraison d'une commande
- afficher les livraisons en cours
- afficher l'historique des livraisons

Une livraison peut contenir les informations suivantes :

- commande concernée
- livreur concerné
- date de livraison prévue
- date de livraison réelle
- statut de la livraison

3.5. Affichage et consultation

L'application doit permettre d'afficher :

- tous les clients
- tous les livreurs
- toutes les commandes
- les commandes en cours de livraison
- les livraisons terminées

4. Fonctionnalités avancées (optionnelles, fortement recommandées)

Chaque binôme devra implémenter au moins 2 fonctionnalités avancées parmi les suivantes :

- tri des clients par nom
- tri des commandes par date
- recherche multicritère
- calcul du nombre total de commandes livrées
- statistiques sur les livreurs les plus actifs
- affichage des commandes d'un client donné
- gestion de plusieurs types de livraison :
    - express
    - standard
- affichage du livreur ayant effectué le plus de livraisons

5. Contraintes techniques

Le projet doit respecter les contraintes suivantes :

- langage imposé : Java
- application en mode console
- utilisation obligatoire de la programmation orientée objet
- programme structuré en plusieurs classes
- utilisation de constructeurs
- respect de l'encapsulation avec attributs privés et accesseurs si nécessaire
- utilisation recommandée de ArrayList pour stocker les objets
- menu interactif avec Scanner

- code clair, indenté et organisé

6. Classes minimales attendues

Le projet devra comporter au minimum les classes suivantes :

Classe Client
Responsable de représenter un client.

Attributs possibles :
- id
- nom
- prenom
- telephone
- adresse
- email

Méthodes possibles :
- afficherDetails()

Classe Livreur
Responsable de représenter un livreur.

Attributs possibles :
- id
- nom
- prenom
- telephone
- vehicule

Méthodes possibles :
- afficherDetails()

Classe Commande
Responsable de représenter une commande.

Attributs possibles :
- id
- client
- description
- dateCommande
- statut

Méthodes possibles :
- afficherCommande()
- modifierStatut()

Classe Livraison

Responsable de représenter une livraison.
Attributs possibles :

- commande
- livreur
- dateLivraisonPrevue
- dateLivraisonReelle
- statut

Méthodes possibles :

- afficherLivraison()
- terminerLivraison()

Classe ServiceLivraison
Responsable de gérer l'ensemble du système.
Attributs possibles :

- listeClients
- listeLivreurs
- listeCommandes
- listeLivraisons

Méthodes possibles :

- ajouterClient()
- ajouterLivreur()
- creerCommande()
- affecterLivraison()
- afficherClients()
- afficherCommandes()
- afficherLivraisons()

Classe Main
Responsable de lancer l'application et d'afficher le menu principal.

7. Livrables attendus

Chaque binôme devra rendre :

a. Code source complet
   - projet Java complet
   - code bien organisé
   - classes séparées

b. Diagramme UML
   - diagramme de classes du projet

c. Rapport écrit
   Le rapport peut contenir :
   - introduction
   - objectifs
   - description des classes

- choix de conception
- fonctionnalités réalisées
- difficultés rencontrées
- captures d'écran console ou exemples d'exécution
- conclusion

d. Démonstration orale
   - Présentation courte du projet avec exécution du programme.

8. Critères d'évaluation

- respect du cahier des charges : **40 %**
- qualité du code et organisation POO : **25 %**
- bon fonctionnement du programme : **20 %**
- rapport écrit : **10 %**
- démonstration orale : **5 %**